

Evcil Hayvan Evreni (Pet Universe) Oyunu Geliştirme Yol Haritası

Bu kılavuz, *Go Pet Care* veya *Lovely Cat* benzeri; nefes alan, göz kırpan, gıdıklanan ve kullanıcıyla etkileşime giren tatlı hayvanların yer aldığı kapsamlı bir mobil/web oyunu sıfırdan nasıl inşa edeceğini adım adım gösteren teknik ve pratik yol haritasıdır.

Temel Araç Seti (Toolchain)

Yol haritasına geçmeden önce bilgisayarına kurman ve öğrenmen gereken araçlar:

1. Oyun Motoru: Unity (LTS Sürümü) (C# Dili) veya Godot Engine (GDScript). *Mobil odaklılık ve Spine desteği için Unity tavsiye edilir.*
2. Animasyon Editörü: Spine 2D (İskeletsel 2D animasyon).
3. Çizim Editörü: Adobe Photoshop, Figma veya Krita (Katmanları Spine'a aktarmak için).
4. Ses Kaynakları: Audacity (Basit ses kesme/düzenleme işleri için).

Adım Adım Geliştirme Süreci

ADIM 1: Konsept Tasarımı ve Oyun Döngüsü (Game Loop)

Kod yazmaya başlamadan önce oyunun kurallarını ve petlerin davranışlarını belirle.

- Yapılacaklar:
 1. Core Loop Çizimi: Karakterin temel döngüsünü kağıt üzerinde tasarla. (Örn: *Petin karnını doyur -> Enerji kazansın -> Mini oyuna sok -> Coin kazan -> Yeni giysi/oda süsü al*).
 2. İhtiyaç Listesi Çıkar: İlk sürümde kaç pet olacak? (Öneri: Başlangıç için sadece 1 kedi veya 1 köpek). Kaç adet ihtiyaç barı olacak? (Öneri: Açlık, Temizlik, Uyku, Sevgi).

ADIM 2: Grafiklerin Katmanlı Hazırlanması (Figma / Photoshop)

Karakterlerin göz kırpayabilmesi ve hareket edebilmesi için parçalanabilir çizilmeleri gerekir.

- Nasıl Yapılır?
 1. Karakterini çizdikten sonra hareket edecek her organı ayrı bir katmana (layer) ayır.

2. Göz bölgesi için: `sol\_goz\_acik`, `sol\_goz\_kapali`, `sag\_goz\_acik`, `sag\_goz\_kapali`, `goz\_bebekleri`.
 3. Ağız bölgesi için: `agiz\_mutlu`, `agiz\_kapali`, `agiz\_acik`.
 4. Gövde için: `kafa`, `kulak\_sol`, `kulak\_sag`, `govde`, `kuyruk`, `sol\_kol`, `sag\_kol`, `ayaklar`.
- Araç Kullanımı: Katmanları tek tıkla Spine formatında dışa aktarmak için Photoshop/Krita içindeki "LayersToSpine" scriptini kullan. Bu script görselleri keser ve Spine'in okuyabileceği bir `.json` koordinat dosyası hazırlar.

ADIM 3: Spine 2D ile Karakter Rigleme ve Animasyon

Karaktere iskelet giydirerek onu hayata döndür.

- Nasıl Yapılır?
1. Setup Mode (Kurulum): LayersToSpine çıktısını Spine'a aktar. Karakterin eklemelerine kemikler (bones) ekle. Örneğin kafaya bir kemik, kulaklara o kemiğe bağlı alt kemikler (child bones) ekle.
 2. Mesh & Weight (Ağ ve Ağırlık): Kıyafet veya gövde gibi esnek alanları yumuşak bükebilmek için görsellere Mesh uygula ve kemiklere ağırlık ver. Bu sayede canavar nefes alırken göbeği yumuşakça şişer.
 3. Temel Animasyonları Hazırla (Animate Mode):
- `idle\_normal`: Karakterin dururken hafifçe nefes alması (kafa ve gövde yavaşça yukarı-aşağı hareket eder, kuyruk sallanır).
 - `blink` (Göz Kırpma): Göz katmanlarını kapatıp açan 0.1 saniyelik hızlı animasyon.
 - `touch\_reaction`: Kullanıcı pete dokunduğunda sevinme veya gıdıklanma hareketi.
 - Dışa Aktarma: Animasyonları oyun motorunda kullanmak için Skeleton (.json veya .skel) formatında dışa aktar. Editör sana bir `.atlas` dosyası, bir `.png` atlas sayfası ve bir veri dosyası verecektir.

ADIM 4: Oyun Motorunda Proje Kurulumu (Unity)

Projeyi oluştur ve Spine paketlerini içeri aktar.

- Nasıl Yapılır?
1. Unity Hub üzerinden yeni bir 2D (URP) projesi oluştur.
 2. Spine Unity Runtime paketini resmi sitesinden indirip projene (Assets klasörüne) sürükle.
 3. Dışa aktardığın Spine dosyalarını (atlas, png, json) Unity içine sürükle. Unity bunları otomatik olarak `Spine SkeletonData Asset` dosyasına dönüştürecek.
 4. Sahnede bir GameObject oluşturup üzerine `SkeletonAnimation` bileşeni ekle ve hazırladığın veriyi bağla. Artık petin ekranda canlı duruyor!

ADIM 5: İhtiyaç Barlarının ve Mekaniklerin Kodlanması (C#)

Petin yaşamasını ve değişken durumlarını yöneten kod bloklarını yaz.

- Nasıl Yapılır?

1. PetState Sınıfı Yaz: Petin değişken değerlerini tutacak bir C# scripti oluştur:

```
public class PetStats : MonoBehaviour {
    public float hunger = 100f;
    public float hygiene = 100f;
    public float love = 100f;

    void Update() {
        // Zamanla barların düşmesi
        hunger -= Time.deltaTime * 0.5f; // 1 saniyede 0.5 birim düşer
        hygiene -= Time.deltaTime * 0.3f;
    }
}
```

2. Dokunma ve Gıdıklama Mekanizması (Touch/Raycast):

Kullanıcı pete dokunduğunda bunu algılamak için pete bir 'PolygonCollider2D' ekle. C# tarafında dokunmayı yakala:

```
void OnMouseDown() {
    // Dokunulduğunda sevinme animasyonunu tetikle
    skeletonAnimation.AnimationState.SetAnimation(0, "touch_reaction", false);
    // Animasyon bitince otomatik olarak idle_normal'e geri dönsün
    skeletonAnimation.AnimationState.AddAnimation(0, "idle_normal", true, 0f);

    love = Mathf.Min(100f, love + 5f); // Sevgiyi artır
}
```

3. Göz Kırpma (Blink) Otomasyonu:

Göz kırpmasının rastgele aralıklarla olması için bir Coroutine yaz:

```
IEnumerator RandomBlinkRoutine() {
    while(true) {
        yield return new WaitForSeconds(Random.Range(2f, 5f)); // 2-5 sn arası bekle
        // Göz kırpma animasyonunu 1. kanalda (track 1) oynat ki ana hareketi bozmasın
        skeletonAnimation.AnimationState.SetAnimation(1, "blink", false);
    }
}
```

ADIM 6: Ses Entegrasyonu ve Görsel Cila (Juice)

Oyunu kuru kod yığınından çıkarıp tatlı ve eğlenceli hale getir.

- Nasıl Yapılır?

1. Dokunma Tepkileri (Feedbacks): Pete dokunulduğunda veya beslendiğinde arkasından tatlı küçük kalpler veya yıldızlar fıskırmasını sağla (Unity Particle System kullanarak 2D konfeti/yıldız efekti oluştur).

2. Ses Efektleri (SFX):

- Buton tıklamaları için "pop" veya "bubble" sesleri.
- Petin sevinç sesleri için tatlı mırıltı/havlama efektleri.
- Arka plana kısık sesli, rahatlatıcı lo-fi veya akustik bir müzik yerleştir.

3. UI Animasyonları (Pop-up): Dükkan veya banyo menüsü açılırken ekranın ortasına `DOTween` kütüphanesi kullanarak elastik bir şekilde büyümesini sağla (`transform.DOScale(1f, 0.3f).SetEase(Ease.OutBack)`).

ADIM 7: Offline Kazanç ve Kayıt Sistemi (Save System)

Oyuncu oyunda değilken de evrenin yaşadığını hissettir.

- Nasıl Yapılır?

1. Zaman Damgası (Timestamp) Kaydı: Oyuncu oyundan çıkarken `DateTime.UtcNow` değerini yerel hafızaya (`PlayerPrefs` veya JSON dosyası) kaydet.

2. Geri Dönüş Hesabı: Oyuncu oyunu tekrar açtığı anda aradaki geçen süreyi hesapla:

```
double secondsPassed = (DateTime.UtcNow - savedExitTime).TotalSeconds;
// Aradan geçen süreye göre açlığı düşür
hunger -= (float)secondsPassed * hungerDecreaseRate;
// Eğer pet dükkanda çalışıyorsa offline coin hesapla
int earnedCoins = (int)(secondsPassed * coinRatePerSecond);
```

3. Kayıt Şifreleme: Kayıt verilerini basit bir XOR şifrelemesiyle koru ki oyuncular coin miktarını kolayca hileyle değiştiremesin.

Kritik Başarı İpuçları (Best Practices)

Önemli İpucu / Dikkat:

Track (Kanal) Yönetimi: Spine animasyonlarını oyun motorunda oynatırken kanalları ayır. Kanal 0'da temel duruş ('idle'), Kanal 1'de sadece göz kırpması ('blink'), Kanal 2'de ise dükkan/banyo yaparken kuyruk sallama gibi bağımsız animasyonları çalıştır. Bu sayede animasyonlar birbirini kesmez, üst üste sorunsuz biner.

Önemli İpucu / Dikkat:

Tüy ve Performans: 2D pet oyunlarında aşırı yüksek çözünürlüklü dokular (textures) mobil cihazlarda aşırı ısınmaya ve RAM tüketimine sebep olur. Pet görsellerini maksimum 2048x2048 boyutunda bir atlas sayfasına sıkıştır ve URP'nin 2D Pixel Perfect kamerasından yararlan.